

Golden Thread — Getting Started

A guided walkthrough for your first session. Five steps, ~10 minutes.

Requirements: Python 3.8+, Claude Code installed. Written against **gt v0.9.6**.

Step 1 — Install

Unzip the plugin and run the installer:

```
unzip golden-thread-plugin.zip
cd golden-thread-plugin
bash install.sh
```

Then **restart Claude Code**. The `/gt:` skills are not available until you do.

The installer sets up two plugins: - **gt** — memory, projects, session workflow, and enforcement hooks - **gt-wiki** — knowledge base with immutable sources

Step 2 — Initialize your vault

Open any Claude Code session and run:

```
/gt:gt-init
```

Claude will ask for: 1. A vault path — an Obsidian vault folder is ideal, but any folder works 2. Your domain/team name — e.g. "Platform Engineering" or "My Team"

The script creates the folder structure, writes `~/.claude/vault-config.json` (the pointer every other skill reads), and **installs Core-rule enforcement hooks** into `~/.claude/settings.json`. Safe to re-run on a machine you've already set up.

If you already have an Obsidian vault, just give `gt-init` that path. It will only add what's missing — your existing notes are untouched.

Step 3 — Create your first project

```
/gt:gt-create
```

Claude will ask for a project name and a quick brain dump of what it is. That brain dump becomes `idea.md` — the immutable origin story for the project.

For a project you're already working on, use:

```
/gt:gt-ingest
```

This scans your existing notes and memory files and imports them into the vault without touching the originals.

Step 4 — Open a project at the start of each session

```
/gt:gt-open <project-name>
```

This reads your project docs in order — `idea` → `research` → `decisions` → `design` → `spec` → `runbook` — reads `CONVENTIONS.md` and `PROTOCOL.md` once per session, then reads the session memory index and stops. It summarizes where things stand and asks where to pick up.

Why it stops at the index: a project with 70 notes would otherwise dump 2,000 lines into context. `gt-open` reads the one-line index and loads individual files only when you need them. If `research.md` exceeds ~200 lines, it reads headings first, then only the relevant sections.

Step 5 — Write back at the end of each session

```
/gt:gt-work
```

This is the step that keeps the vault alive. Claude will write back what was learned:

DESTINATION	WHAT GOES THERE
<code>research.md</code>	New findings and gotchas, dated
<code>decisions.md</code>	Architectural choices (ADRs)
<code>design.md</code>	Updated architecture if it changed
<code>runbook.md</code>	Operational commands and landmines
<code>memory/*.md</code>	Session state

`gt-work` also checks whether any findings should be promoted wider (to `Knowledge/` or `global-memory/`) and asks you before writing. It updates the project's `README.md` frontmatter properties (`stage` , `topology`) if the project changed phase.

Skipping `gt-work` is how the vault decays. Facts that aren't written down are forgotten by the next session.

Typical session pattern

```
/gt:gt-open my-project    ← load context, see where you left off

# ... work happens ...

/gt:gt-work               ← write back findings before closing
```

Periodic maintenance

```
/gt:gt-lint               ← find broken links, orphaned pages, unlisted memory files, unen
/gt:gt-promote            ← graduate a finding that proved true across projects
/gt:gt-refresh            ← check if any source documents changed upstream
/gt:gt-review             ← pull uncaptured items from Obsidian daily notes
/gt:gt-settings           ← view and toggle what Golden Thread does automatically
/gt:gt-farm               ← route bulk or mechanical tasks to an external AI service as a
/gt:gt-validate           ← independently verify a claim before recording it as fact
```

Adding a knowledge base (gt-wiki)

If you want a structured wiki alongside your project memory:

```
/gt:gt-wiki-init
```

Then add sources:

```
/gt:gt-wiki-ingest <url or file path>
```

Query it:

The wiki uses immutable sources — every ingested document is stored verbatim and never modified. Knowledge pages are synthesized summaries that link to the originals.

Key concepts in one paragraph

The vault is organized by **how widely a fact applies**. Session findings go in `memory/`. Settled discoveries go in `research.md`. Choices that shaped the architecture go in `decisions.md`. Facts true across multiple projects go in `Knowledge/`. Facts needed in every session everywhere go in `global-memory/`. Above all of these sits the **Core-rule tier**: rules enforced via hooks wired into `~/.claude/settings.json`, not merely stored as files. `gt-init` installs these hooks automatically. Promote a fact when a second project proves it general — not before. When unsure, file it narrow; promotion is cheap, demotion isn't. One more principle: **claims should be verified by re-deriving them, not by reviewing the reasoning that produced them** — a reviewer who sees your reasoning inherits your blind spot. Use `/gt:gt-validate` before recording any finding that matters.

Troubleshooting

Skills not showing up — restart Claude Code after install.

"vault-config.json not found" — run `/gt:gt-init`.

A rule seems ignored — `CLAUDE.md` is read at session start; rules added mid-session apply from the next session.

Memory files aren't loading — that's by design. Ask for a file by name, or run `/gt:gt-lint` to check if it's unlisted.

Core rules seem to apply sometimes but not always — run `/gt:gt-lint` and look for `core-unenforced` findings. The hook may not be wired. Fix:

```
python3 <plugin>/scripts/vault_init.py install-core-rules --vault <vault>
```

I want to turn off the automatic session report — run `/gt:gt-settings set report_card off`.

Timestamp missing from responses — the enforcement hook is unwired. Run `/hooks` in Claude Code or restart. If it persists, re-run `install.sh`.

For the complete skill reference, see [MANUAL.md](#).